Project Planning
Project Time Management

Overview

- *Project time management* is the project management knowledge area concerned with *analyzing the logical and temporal relationships* among the activities needed to complete the project
- Three elements form the core of time management analysis:
  - The *schedule data* and associated calculations (e.g., activity definitions and estimates)
  - The *scheduling method* applied to the schedule data in order to define estimated start and end dates for activities and milestones, including project completion
  - The *schedule* that represents the output from a scheduling tool applying the method to the data
- In a conventional methodology, the project schedule acts as the planning backbone for virtually all other project activities

Project time management processes

- The project time management processes include:
  1. *Define activities*. Identifies specific activities to be carried out in work packages
  2. *Sequence activities*. Identify and document relationships among activities
  3. *Estimate activity resources*. Estimate the number and type of resources needed for activity, such as staff, materials, equipment, software, etc.
  4. *Estimate activity durations.* Estimate number of work periods (e.g. hours, days, weeks) to complete activity with estimated resources
  5. *Develop schedule*. Analyze network of activity sequences, durations, resources, and constraints to estimate planned dates for activities and milestones
  6. *Control schedule*. Monitor schedule status and manage schedule updates

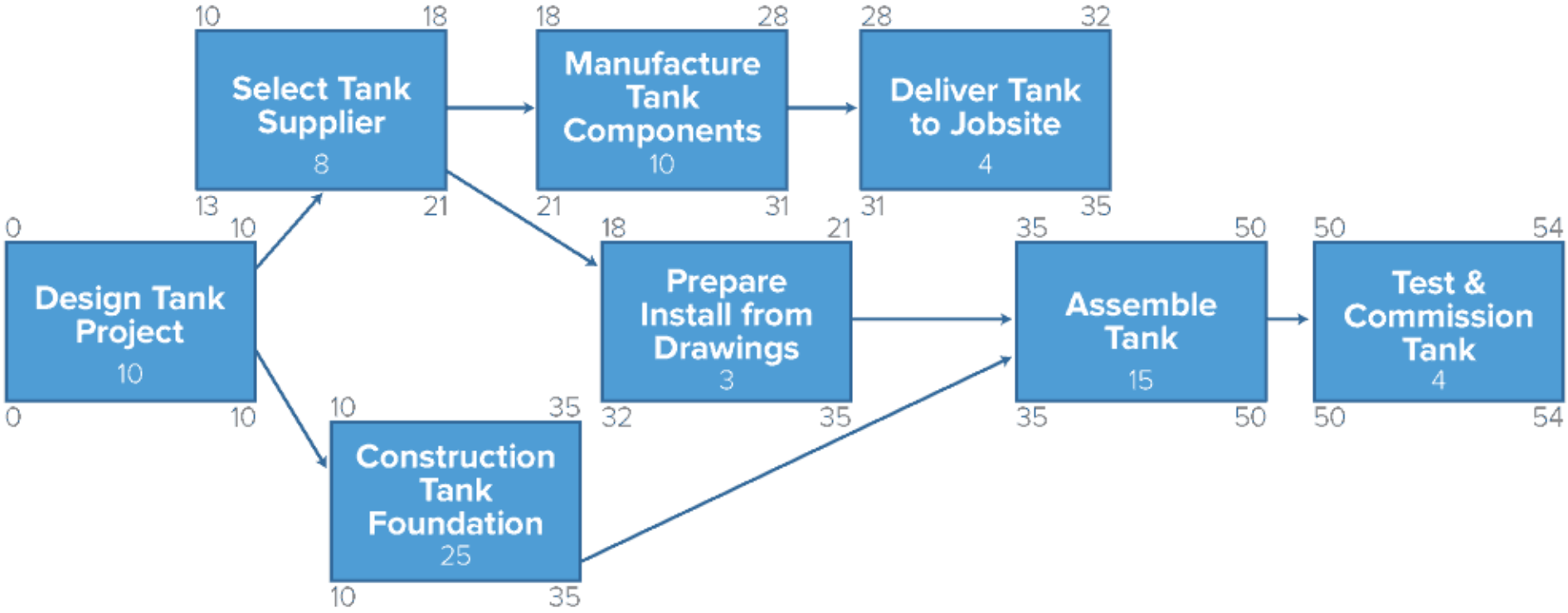
Scheduling workflow

- Define activities
  - Use of WBS helps guide definition process and organize activities
- Perform activity sequencing
  - Develop schedule framework according to what is *logically possible* – perform resource allocation later
- Estimate effort – the total number of labor units (*e.g.* staff-days) for each activity
- Estimate elapsed time
- Identify resources for each activity
- Apply calendars to schedule framework

Scheduling workflow

- Estimate activity duration based on resources for activity
- Perform *forward pass* or *backward pass critical path* analysis to generate schedule model
- Apply schedule compression, if needed
- Perform ‘what-if’ scenario analysis to identify contingency and risk response needs
- Apply resource leveling to schedule model

Scheduling workflow



Planning, Estimating, Scheduling

- What's the difference?
  - **Plan:** Identify activities. No specific start and end dates.
  - **Estimating:** Determining the size & duration of activities.
  - **Schedule:** Adds specific start and end dates, relationships, and resources.

How To Schedule

- Identify "what" needs to be done
  - Work Breakdown Structure (WBS)
- Identify "how much" (the size)
  - Size estimation techniques
- Identify duration
  - Effort estimation techniques
- Identify the dependency between tasks
  - Dependency graph, network diagram
  - Gantt chart and PERT
- Estimate total duration of the work to be done
  - The actual schedule
  - PERT and CPM

PERT and CPM

What does It Mean?	
PERT – PERT is a popular project management technique that is applicable when the time required to finish a project is not certain	CPM – CPM is a statistical algorithm which has a certain start and end time for a project
Model Type	
PERT – PERT is a probabilistic model	CPM – CPM is a deterministic model
Focus	
PERT – The main focus of PERT is to minimise the time required for completion of the project	CPM – The main focus of CPM is on a trade-off between cost and time, with a major emphasis on cost-cutting.
Orientation type	
PERT – PERT is an event-oriented technique	CPM – CPM is an activity-oriented technique

Overview

- The *Define Activities* process identifies the specific actions or tasks needed to carry out the project and produce the project deliverables
- In a conventional project methodology, the *Create WBS* process defines the project deliverables and work packages
  - In an adaptive/agile project methodology, only the high-level deliverables are defined up front, while work packages are defined at the iteration level

Activity definition

- ***Decomposition*** is the process of breaking the project scope and deliverables into smaller, more manageable components
- Decomposition is usually performed in a top-down, hierarchical manner
- Decomposition proceeds until it is possible to define the component as a *work package*:
  - A deliverable or work component at the lowest WBS level that includes schedule activities and milestones to complete the deliverables or work
- Significant part of the WBS decomposition can be defined by WBS templates

Sidebar: A useful rule of thumb

- No work package should have a duration greater than four to six weeks
- For knowledge work, durations should be in the range of one to three weeks, because knowledge work is harder to track than tangible work
  - People tend to ***back-end load*** tasks with lengthy durations by pushing all the effort toward the end

Activity definition workflow

- Choose an appropriate WBS template as a guide for activity definition
- Enter WBS template into a project management tool (if preloaded template not available)
- Define activities in the task list of the project management tool
- If decomposition results in an activity with a deliverable/work component and a duration of 1-2 weeks, consider it a work package
- If decomposition results in a set of shorter (<1 wk) activities, group them into one or more 1-2 week work packages that produce deliverable/work components

Activity definition workflow

- Assign each activity a unique identification code that remains with the activity if it is moved in the list
  - The WBS code is a position-dependent hierarchical code – it does not stay with an activity if moved
  - Example: Architectural Cost-Benefit Analysis has a WBS code of 4.2.1.2 but a unique identification code of (hypothetically) ADM2 (or 25.0, or any other code)
  - 4.2.1.2: **ADM2** – Architectural Cost-Benefit Analysis
- Work packages usually have additional activity detail specified in their WBS dictionary entry

Agile Perspective: Define Activities

Essential, but do it just in time

- "The activity list is a comprehensive list that includes all schedule activities required on the project"
- In an adaptive/agile project, this list cannot—and should not—be generated early in the project
- Attempting to define activities up-front in a complex project leads to wasted effort and inconsistencies between documents and reality
- Instead, application of the agile method delays specific activity definition to the start of each individual sprint, and delegates the activity definition to the individual sprint development teams

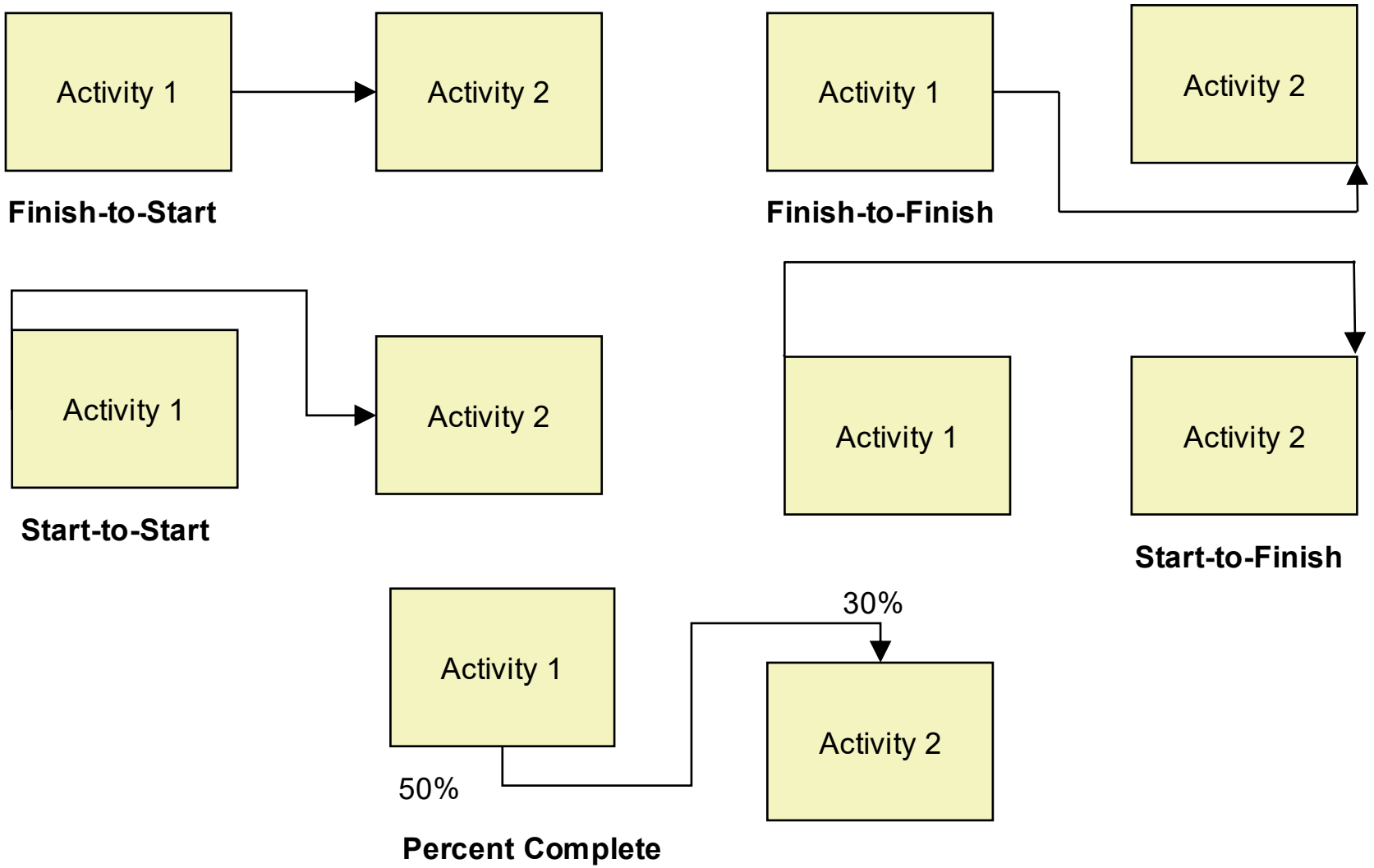
Activity Sequencing: Introduction

- *Activity sequencing* identifies and documents the ***logical relationships*** among activities in a project
- Logical sequencing is determined by precedence (predecessor-successor) relationships
- Essential inputs
  - Activity list, which may be developed concurrently with WBS
  - May use scope statement information to determine or refine precedence constraints
- Essential outputs
  - Project schedule network diagrams
  - Updated activity list/WBS

Precedence Diagram Method (PDM)

- The Precedence Diagram Method (PDM) is a graphical schedule diagramming method aka Network Diagram
- Represents activities as rectangular nodes
- Connects the nodes with arrows to show dependencies
- Connection points of arrows to activities refine and impose constraints on dependencies
- Classified as an '*Activity on Node*' (AON) diagramming scheme
  - Alternative is '*Activity on Arrow*' (AOA) that models project in states, with activities as transitions from one state to another

Dependency types illustrated



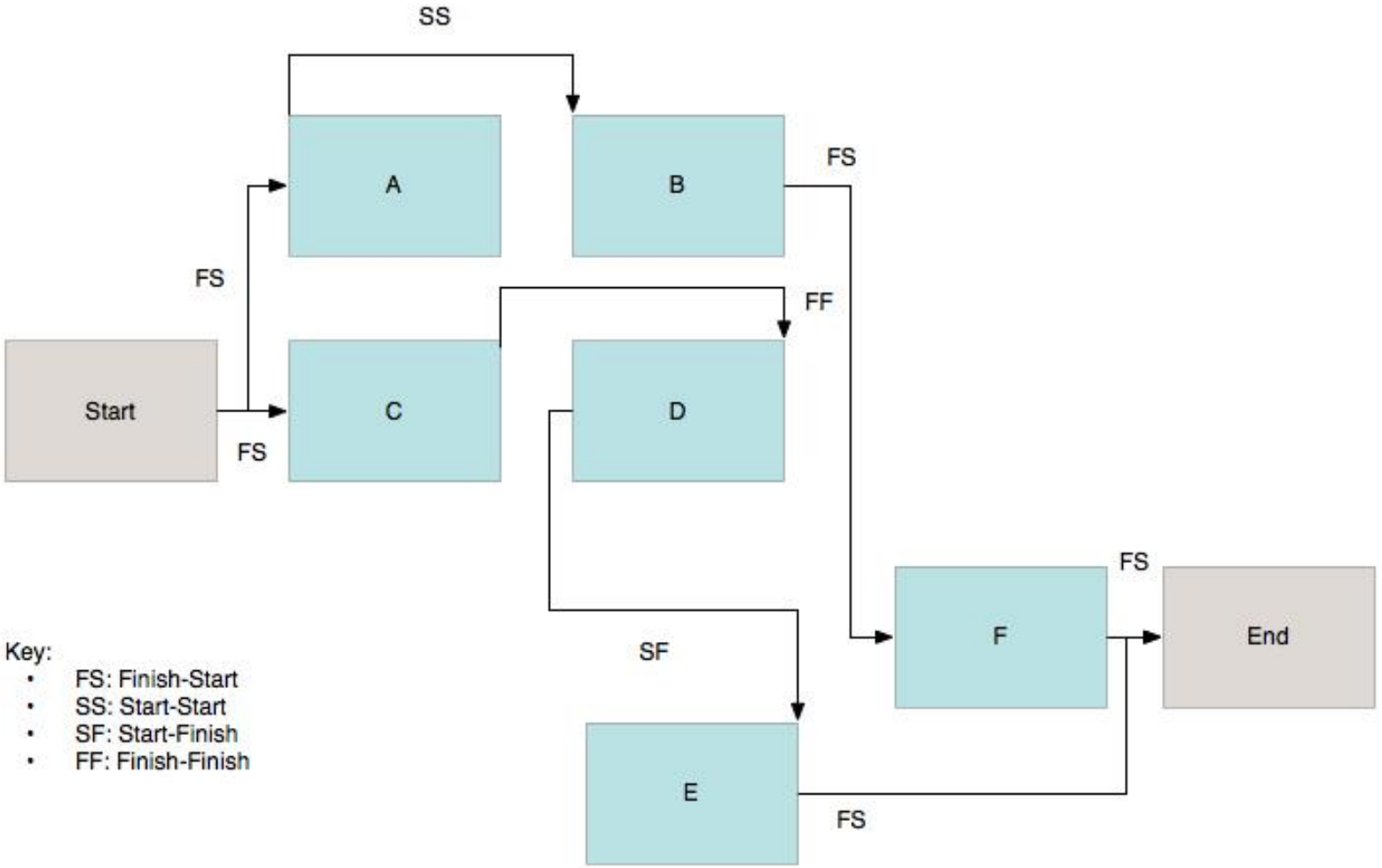
Dependency types

- ***Finish-to-start***. Start of successor activity depends on finish of predecessor activity
  - ***Example:*** Start of testing after code completion in traditional waterfall development
- ***Finish-to-finish***. Finish of successor activity depends on finish of predecessor activity
  - ***Example***: Acceptance of a component can only complete when acceptance of last subcomponent is complete
- ***Start-to-start***. Start of successor activity depends on start of predecessor activity
  - **Example**: Start of acquisition of third-party software component triggers training for involved developers

Dependency types

- ***Start-to-finish***. Finish of successor activity depends on start of predecessor activity
  - *Example:* Subcontract *x* will complete *t* days after subcontract *y* begins
- ***Percent complete***: Last *n*% of successor activity depends on *m*% completion of predecessor activity
  - *Example:* Last 30% of network interface development will begin when 50% of application development is complete
- ***Note:*** A better choice of terms might be ***dependent*** and ***independent*** activities, as in the cases of ***Start-to-start*** and ***Start-to-finish***

PDM (Precedence Diagram Method): example



Activity sequencing

	WBS	Name	Predecessors
1	**1**	**− Automatic Toll Booth System**	
2	**1.1**	**+ Management**	
20	**1.2**	**+ Environment**	
33	**1.3**	**+ Requirements**	
42	**1.4**	**− Design**	
43	1.4.1	Inception phase architecture prototyping	
44	**1.4.2**	**− Elaboration phase architecture baselining**	
45	**1.4.2.1**	**− System architecture definition**	
46	1.4.2.1.1	Define high-level hardware architecture	43
47	1.4.2.1.2	Define high-level communications architecture	43,46SS+5 days
48	**1.4.2.2**	**− Architecture design modeling**	
49	1.4.2.2.1	Perform ABD (architecture-based design)	45
50	1.4.2.2.2	Perform quality attribute analysis (ATAM)	49
51	1.4.2.2.3	Perform cost/benefit analysis (CBAM)	50
52	1.4.2.2.4	Document software architecture	51
53	**1.4.2.3**	**− Design demonstration planning and conduct**	
54	1.4.2.3.1	Select architectural baseline critical use-cases	48
55	1.4.2.3.2	Select architectural baseline critical functionality	54
56	1.4.2.3.3	Design architectural baseline components	55
57	**1.4.3**	**+ Construction phase design modeling**	
60	1.4.4	Transition phase architecture design maintenance	
61	**1.5**	**− Implementation**	
62	1.5.1	Inception phase component prototyping	
63	**1.5.2**	**− Elaboration phase component implementation**	
64	**1.5.2.1**	**− Critical component coding demonstration integration (architectural baseline)**	
65	1.5.2.1.1	Implement architectural baseline components	56,62
66	1.5.2.1.2	Integrate architectural baseline components	65FF
71	**1.5.3**	**+ Construction phase implementation**	
72	1.5.4	Transition phase component maintenance	
73	**1.6**	**− Assessment**	
74	1.6.1	Inception phase assessment planning	
75	**1.6.2**	**− Elaboration phase assessment**	
76	1.6.2.1	Select and define architectural baseline tests	53
77	1.6.2.2	Implement architecture test scenario	76,66
78	1.6.2.3	Test architectural baseline	77

Note dual predecessors. Default relationship is Finish-to-Start. Here, we have defined a Start-to-Start relationship with an added lag of 5 days

Here, we have defined a Finish-to-Finish relationship: this is common for implementation/integration task pairs

Leads and lags

- ***Lead***. A lead moves an activity back in time from its expected point. Sometimes referred to as an accelerated activity
  - *Example:* Beginning work on second draft of document before first draft is complete
- ***Lag***. A lag introduces a delay into a successor activity. Restricts the start of the successor, even if predecessor activity is complete
  - *Example:* Requiring ten days lag before acceptance testing can begin (possibly introduced as contingency)

Outputs

- *Schedule network diagrams*. Schedule network diagrams graph the project activities and their dependencies. These may be produced manually or using project management software
  - Documentation explaining discretionary dependencies, leads and lags, or other exceptional dependencies should accompany the diagram
- *Project document updates*. The Sequence Activities process may lead to updates in the following project documents:
  - Activity lists
  - Activity attributes, specifically the predecessor and successor activities, logical relationships, and leads and lags attributes
  - Milestone lists
  - Risk register

Task

- Name
- ID
- Description of work
- Duration (days)
  - Start Date (Earliest, Latest)
  - Finish Date (Earliest, Latest)
- Resources (People and equipment)
  - Effort (In staff-days)
- Predecessors (other tasks)
- Inputs (documents, decisions, information)
- Successors (other tasks)
- Outputs (documents, decisions, information)

Agile Perspective: Sequence Activities

Useful concepts, but apply them just in time

- The concepts of activity dependency are applicable across all project management methodologies
- However, in an adaptive/agile project, the activity list, upon which sequencing depends, is generated in small chunks throughout the project at the start of each individual sprint
  - This is the logical time at which to work out sequencing of specific activities
  - Define activities no earlier than during iteration planning